

LAB SESSION ON TRAFFIC LIGHTS

DESIGN OF TRAFFIC LIGHT CONTROLLERS IN VHDL

ESTIMATED PREPARATION TIME: 4-6 HRS

This lab description is derived version of the according lab description of Prof. Dr. Leutelt.

OBJECTIVE AND INTRODUCTION

In this fourth lab we develop a traffic light controller for an intersection board. It is our first implementation, in which we go through all the steps of development process: from specification of the FSM, via modelling by VHDL to implementation and test on the FPGA.

Two different designs will be implemented:

- first, a simpler version with an FSM with symbolic states and light phases of *equal* duration and
- a second version using an additional counter where the duration of light phases can be individually adapted (timed state machine).

SPECIFICATION OF THE TRAFFIC LIGHT CONTROLLER

The west-east direction is the main direction, the north-south direction the secondary road. At night the lights follow a different sequence than at day. The sequence shall be selected by a switch of an IOM board.

The sequences and durations of light phases for night and day configuration are specified by the tables below.

- An empty table entry corresponds to a LOW signal value, otherwise the signal shown is HIGH.
- Phase 0 is the reset state after an asynchronous reset.
- In regular day operation, the sequence from phase 1 to phase 8 (= a cycle) is repeated continuously, at night the controller alternates between phase 2 and 3.
- When a change of the day/night control signal occurs, it has no immediate effect and the current cycle is finished first.
- A change of the control signal from day to night forces a transition from phase 8 (day) to phase 1 (night), and a change from night to day forces a transition from phase 3 (night) to phase 0 (day).

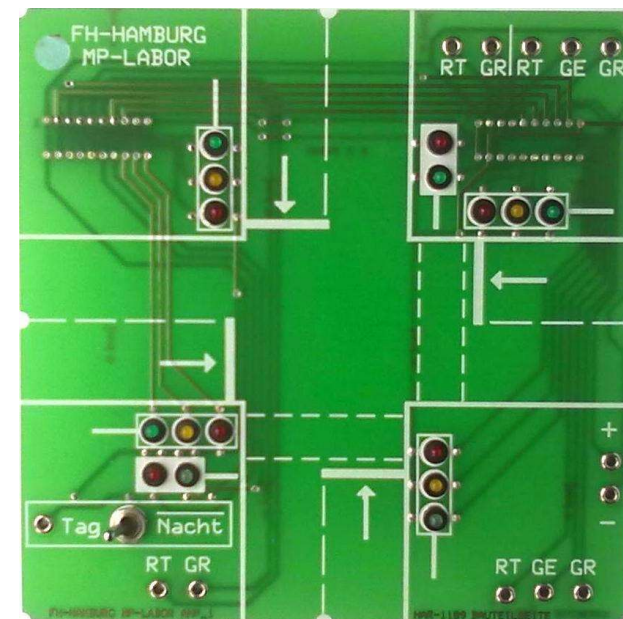


Figure 1: Intersection board used in this lab task. (to be connected via IOM-Board)

PART 1: IMPLEMENTATION OF THE CONTROLLER WITHOUT MODELLING OF DURATIONS (SYMBOLIC STATES)

In the following, the controller shall be developed assuming equal durations of the traffic light phases. The FSM shall be of Moore type and the states shall be coded by symbolic states.

Table 1: Sequence and duration of the traffic phases (day time).

Phasen <i>phases</i> (Tag <i>day</i>)		0	1	2	3	4	5	6	7	8	Continue with phase 1...
Dauer <i>duration</i> (ticks)		2	2	1	8	1	2	1	4	1	
Hauptrichtung <i>main direction</i>	Autos <i>cars</i>		RT	RT			RT	RT	RT	RT	
		GE		GE		GE					
					GR						
	Fußgänger <i>pedestrians</i>	RT	RT			RT	RT	RT	RT	RT	Legende <i>legend</i> :
				GR	GR						RT = rot <i>red</i>
Seitenstraße <i>secondary road</i>	Autos <i>cars</i>		RT	RT	RT	RT	RT	RT			GE = gelb <i>amber</i>
		GE						GE		GE	GR = grün <i>green</i>
									GR		
	Fußgänger <i>pedestrians</i>	RT	RT	RT	RT	RT	RT			RT	
								GR	GR		

Table 2: Sequence and duration of phases during night time.

Phasen <i>phases</i> (Nacht <i>night</i>)		0	1	2	3	Continue with phase 2...
Dauer <i>duration</i> (ticks)		2	2	1	1	
Hauptrichtung <i>main direction</i>	Autos <i>cars</i>		RT			
		GE				
	Fußgänger <i>pedestrians</i>	RT	RT			
Seitenstraße <i>secondary road</i>	Autos <i>cars</i>		RT			
		GE		GE		
	Fußgänger <i>pedestrians</i>	RT	RT			

PREP TASK 1.1:

- Draw the block diagram with all port signals of the traffic light controller. (Without internal structure of block).

PREP TASK 1.2:

- Draw the complete state diagram (FSM) for the traffic light controller (including day and night operation) with states, transitions and output signals.
- Why is Mealy behavior not required for this task?

PREP TASK 1.3:

- Write a VHDL model of the traffic light controller with symbolic states. (attention: symbolic states cannot be displayed in EPWave)
- Write 5 test cases for the verification.
- Verify the correct function of the circuit by simulation (give the do file or testbench source and simulation result).

LAB TASK 1.1

Synthesize the VHDL model of the traffic light controller and implement it on the FPGA.

- Verify the correct operation of the FPGA with the test cases from PREP TASK 1.3 and document the results in the report.
- Measure and display the three control signals for the lights RT, GE, GR of the secondary road with an oscilloscope for one complete day sequence plus the transition to the night sequence.
- List the allocated resources given in the implementation report.

- Analyze the synthesis/implementation report and realize which state encoding was used for the states.
- Change RESET from asynchronous to synchronous and analyze different resource usage.

LAB TASK 1.2

Analyze the synthesized and implemented circuit schematics.

- Extract the gate realization of the next-state logic and the output logic and draw the circuit.

PART 2: IMPLEMENTATION OF THE CONTROLLER WITH DURATION MODELLING (TIMED STATE MACHINE)

MODELLING OF TRAFFIC LIGHT PHASES BY A COUNTER

Since the durations of traffic light phases vary ('amber' is typically shorter than 'red' or 'green', the phases of the main road longer than those of the secondary road), durations of the phases need to be modelled. This is typically achieved by a synchronous modulo-n counter (see Figure 2) with a clear input (here: CLRN). The idea is that a transition from one state to the next is only done when a previously defined count value is reached.

The clearing of the counter can be either

- (1) done after *every* state transition or, alternatively,
- (2) when a complete traffic light sequence is completed.

The advantage of the latter approach is that the counter is less often cleared, whereas the advantages of the first are that the counter is smaller and that the exit value of a certain

state does not depend on values of previous states (since after every state transition the counting starts with 0).

Note: There is a difference between the clear and the reset input. The reset ARESETN is asynchronous and is asserted only once when the traffic light controller is started. The clear input CLRN is synchronous and can e.g. clear the counter after every state transition. The clear input is like a counter preset function that presets the counter with the fixed value 0.

The implementation of the traffic light controller with a counter shall be done according to a “hierarchical design” approach.

STRUCTURE OF THE TRAFFIC LIGHT CONTROLLER (WITH COMPONENTS)

Hint: structural VHDL modelling is explained at the end of this lab description (APPENDIX)

The block diagram in Figure 2 shows how the FSM of the traffic controller and counter communicate with each other. Both parts shall be modelled as individual entities first and can, initially, be simulated and verified in parallel (in separate files and projects) and later instantiated as components in one top-level module as described above.

Hint: The next three PREP TASKS can be prepared more or less independently. Use this fact and split the task within your team to efficiently and effectively prepare the task.

PREP TASK 2.1: Development of the modulo-n counter.

Let the counter be specified as follows:

- The counter is a modulo-n counter that counts upwards to a maximum value (which you must find yourself). The maximum value shall be defined as a *generic* parameter in the entity declaration.
- The counter can be synchronously cleared by a CLRN input.
- The counter has an asynchronous active-low reset and a clock input.
- The counter shall be designed as an entity that will be instantiated as a component in the traffic light controller design.
- Develop a VHDL model for the counter specified above with clear input.

PREP TASK 2.2: Development of the traffic light control FSM.

Make a copy of the project developed in PREP TASK 1.3 and adapt the VHDL model of the simple traffic light controller in a way

- that it can interact with the counter via the signals CNT and CLRN and
- that the state transitions are taken based on the value of CNT.

It is left to you to decide whether you use the counter for

- option (1): FSM clears counter with every state transition) or
- option (2): FSM clears counter once per entire traffic light cycle.
- Is the output CLRN that clears the counter of Mealy or Moore type? Explain why.
- Draw the state diagram for the FSM with duration modelling (state transitions now depending on signal CNT, too).
- Write the VHDL model of the counter and verify it by simulation. Modify the test cases to map the new case.

PREP TASK 2.3: Combine the traffic light control FSM and the counter in one VHDL model.

- Write one VHDL model in which the two submodules are instantiated as components.
- Depending on the chosen option (1) or (2) in PREP TASK 2.2, please answer: Which value do you choose for n (the maximum number of count values) of the modulo-n counter and why? What is the width of the output signal CNT (give reason for your answer)?
Hint: The value given in Figure 2 might differ from the one needed in your implementation.
- Simulate and verify the traffic light control with counter. Adapt the test bench developed in PREP TASK 1.3

LAB TASK 2.1 Implementation of the traffic light controller with counter.

- Implement the traffic light controller with counter on the FPGA.
- Display three traffic light signals which you consider as relevant to show the right operation and durations together with the CLK signal on the oscilloscope. Also include the change between day and night.
- Compare the allocated resources to the traffic light controller without counter.

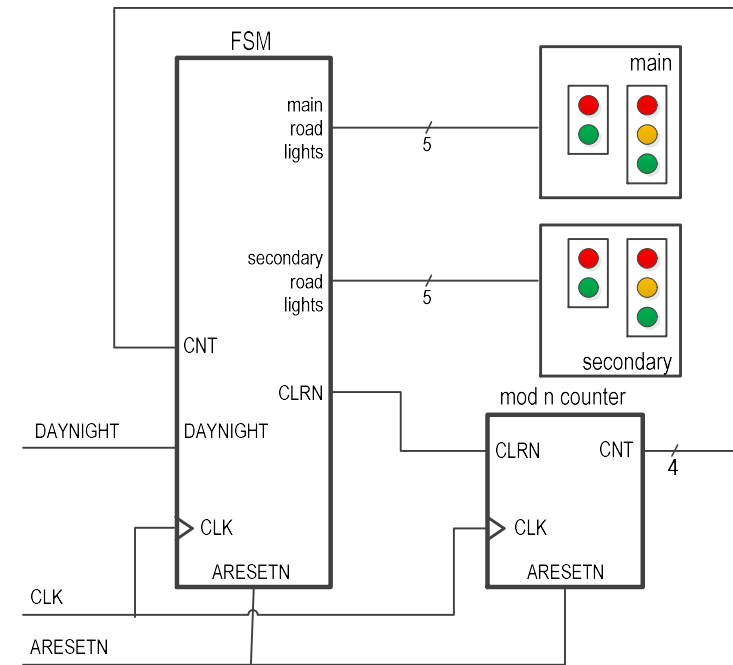


Figure 2: Block diagram of the traffic light controller with a counter for modelling of state durations.

APPENDIX: IN-DEPTH TECHNICAL INFORMATION

HIERARCHICAL DESIGN BY VHDL COMPONENTS (STRUCTURAL VHDL)

Larger designs require a partitioning of the functionality and a hierarchical design approach to make the complexity manageable. “Hierarchical design” means that modules of larger complexity are composed of smaller modules. Each module, referred to as a “component” is described by an entity and an architecture (see Figure 3).

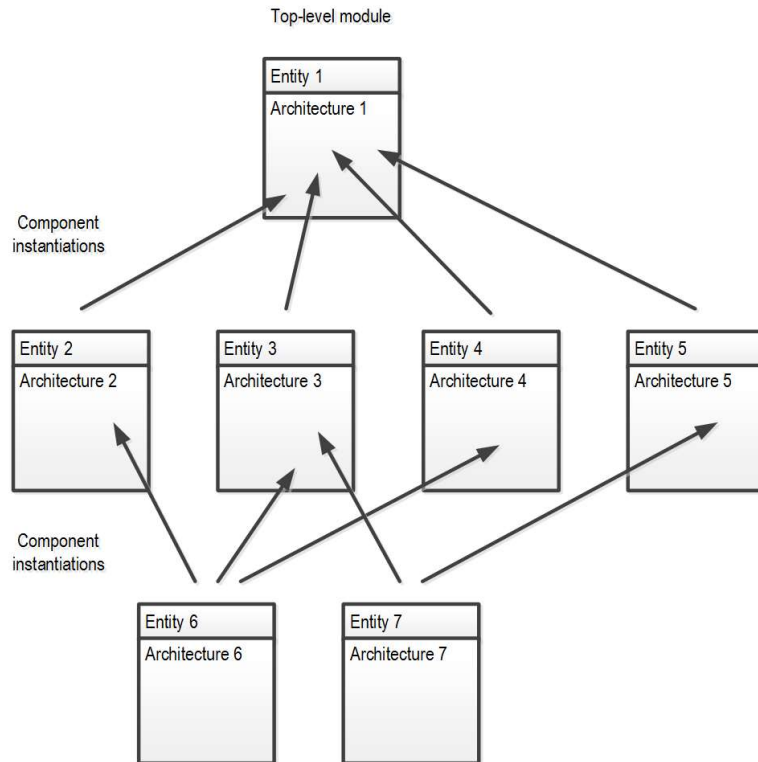


Figure 3: Hierarchical design with VHDL components

Each Entity/Architecture is placed in a separate .vhd-file which needs to be added to the EDAPlayground or Xilinx Vivado project together with the top level entity. Each Entity has to be compiled separately from the bottom-level of the hierarchy to the top-level. If it is

done the other way round, changes of a lower-level entity might lead to compiler errors as they are not yet known at the time of compilation of a higher-level module.

A lower-level component is instantiated in a higher-level architecture

- by a declaration of the component in the declaration part of the top-level architecture and
- by connecting the signals of the top-level entity to the ports of the component (see Figure 4)

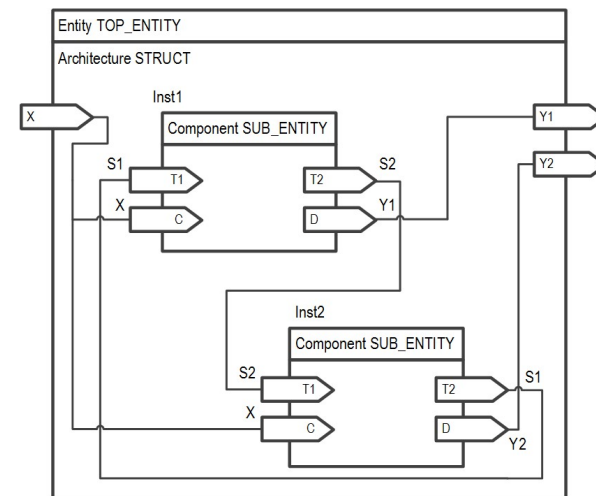


Figure 4: Instantiation of VHDL components within an architecture.

Let the entity of a submodule SUB_ENTITY be declared as follows:

```
entity SUB_ENTITY is
    generic (N : integer :=4);
    port(T1 : in bit;
         T2 : out bit;
         C : in bit_vector(N-1 downto 0);
         D : out bit_vector(N-1 downto 0));
end entity SUB_ENTITY;
```

This module can be instantiated as a component in another module TOP_ENTITY as follows:

```

entity TOP_ENTITY is
    generic (L : integer :=8);
    port(X : in bit_vector(L-1 downto 0);
        Y1 : out bit_vector(L-1 downto 0);
        Y2 : out bit_vector(L-1 downto 0));
end entity TOP_ENTITY;

architecture STRUCT of TOP_ENTITY is

    signal S1,S2 : bit;

    -- declaration of component
    component SUB_ENTITY
        generic (N : integer :=4);
        port(T1: in bit;
            T2: out bit;
            C : in bit_vector(N-1 downto 0);
            D : out bit_vector(N-1 downto 0));
    end component SUB_ENTITY;

begin
    -- instantiation 1 of component
    INST1 : SUB_ENTITY
        generic map (N=>L)
        port map (C=>X,T1=>S1,T2=>S2,D=>Y1);

    -- instantiation 2 of component
    INST2 : SUB_ENTITY
        generic map (L)
        port map (S2,S1,X,Y2);

end architecture STRUCT;

```

The declaration of the component has the same syntax as the entity declaration except for that the keyword “entity” is replaced by “component” and the “is” has been cancelled:

```

component component_name
    generic (generic_list);
    port (port_list);
end component;

```

The connection between the port signals of the submodule with the signals of the top-level entity is done by “port map”, the mapping of the generic parameters by “generic map”:

```

instance_label: component_name
    generic map (generic_association_list)
    port map (port_association_list);

```

In the example above, two instances of the SUB_ENTITY are created (compare to Figure 4). The default value of the internal generic parameter N is in both cases overwritten with the value of parameter L of the top-level entity.

For instance INST1 the named association (component_port=>architecture_signal) is chosen for instantiation of the component, i.e. signals and generics of sub-module and top-level module are listed as pairs and can be mapped in arbitrary order. Instance INST2 is created using the positional association, where the order of signals and generics in the list of top-level and sub module have to match. The first way to instantiate is more descriptive, the second shorter. The following example shows how the N-bit adder could have been implemented using components:

```

entity NBIT_ADDER is

    generic (N : integer := 8);
    port (A : in bit_vector (N-1 downto 0);
        B : in bit_vector (N-1 downto 0);
        C_IN : in bit ;
        SUM : out bit_vector (N downto 0));

end entity NBIT_ADDER;

architecture STRUCTURAL of NBIT_ADDER is

    component FULL_ADD is
        port(X,Y,CIN : in bit;
            SUM,COUT : out bit);
    end component FULL_ADD;

    signal RC : bit_vector (N downto 0);

begin

    RC(0) <= C_IN;

    NBIT : for I in 0 to N-1 generate
        FA1 : FULL_ADD
            port map (X=>A(I),Y=>B(I),CIN=>RC(I),SUM=>SUM(I),COUT=>RC(I+1));
    end generate NBIT;

    SUM(N) <= RC(N);

end architecture STRUCTURAL;

```